Script-Controlled ACL – Restrict Record Access Based on Field Value

Date	29 September 2026
Team ID	SWTID-2026-8067
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	Documentation

Project Description

This project demonstrates how ServiceNow Access Control Lists (ACLs) can be used to protect records according to user roles and field values. A custom **Institution Details** table named **u_institution_details** is created with student, faculty, branch, contact and description fields. The security model uses a script-controlled READ ACL together with separate CREATE, WRITE and DELETE ACLs.

The READ rule allows administrators full access and allows users with the **bb1** role to read records for the EEE branch. Separate operation roles are used for CREATE (**bb2**), WRITE (**bb3**) and DELETE (**bb4**). User impersonation is used to verify authorized and unauthorized access.

Scenarios

Scenario	Description
Scenario 1	An EEE user with bb1 opens Institution Details. The user should be able to read EEE records while records for ECE and CSE remain restricted by the READ ACL.
Scenario 2	A user without bb1 attempts to open the protected list. The READ ACL should deny access to the protected records.
Scenario 3	A user with bb1 and bb2 creates a new Institution Details record. CREATE access is evaluated separately from READ access.
Scenario 4	A user with bb1, bb2 and bb3 edits an allowed record. WRITE access is controlled by the bb3 role.
Scenario 5	A user with bb1, bb2, bb3 and bb4 deletes an allowed record. DELETE access is controlled by the bb4 role.
Scenario 6	An administrator opens the table and retains full access regardless of Branch value.

Technical Architecture

Date	29 September 2026
Team ID	xxxxxx
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	Documentation

Layer	Component	Responsibility
User / Presentation	ServiceNow List & Form	Users access Institution Details records through the ServiceNow interface.
Authentication / Identity	User + Roles	EEE User and custom roles bb1–bb4 determine which ACL checks can succeed.
Security / Logic	ACL Engine + Server-side Script	READ checks administrator role, bb1 and Branch = EEE; CREATE/WRITE/DELETE use separate role requirements.
Data	u_institution_details	Stores Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description.
Testing	Impersonation / ACL Verification	Admin impersonates users and verifies list, form and CRUD behavior.

Architecture Flow

ServiceNow User → Authentication & Roles → ACL Evaluation → u_institution_details Table → Allow / Deny Record Operation

Note: The project uses a ServiceNow custom table rather than a separately managed external database. The security decision is enforced by the ServiceNow ACL engine.

Pre-requisites

• ServiceNow instance with administrator access.

• Ability to elevate to the security_admin role for ACL configuration.

• Knowledge of User Administration, Roles, System Definition → Tables and System Security → Access Control.

• Permission to create custom tables, fields, users, roles and ACLs.

• Test records containing EEE, ECE and CSE branch values.

Project Workflow

Milestone	Focus	Activities
Milestone 1	User and Role Setup	Create EEE User and custom roles bb1, bb2, bb3 and bb4; assign roles as required.
Milestone 2	Institution Details Data Model	Create u_institution_details and add the required fields; populate test records.
Milestone 3	READ ACL	Create advanced record READ ACL; use administrator and bb1 checks with Branch = EEE.
Milestone 4	CREATE / WRITE / DELETE ACLs	Create operation-specific ACLs requiring bb2, bb3 and bb4 respectively.
Milestone 5	Security Verification	Impersonate users and verify allowed and denied operations.
Milestone 6	Outcome & Documentation	Record the configuration, verification results and security behavior.

Milestone 1: User and Role Setup

Date	29 September 2026
Team ID	xxxxxx
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	Documentation

User Creation

Create a test user through User Administration → Users. The project specifies User ID **EEE User**, First name **EEE**, Last name **User**, and email **eeeuser@gmail.com**.

Role Creation

Create custom roles **bb1**, **bb2**, **bb3** and **bb4**. Assign the roles to the EEE User according to the test sequence.

Milestone 2: Institution Details Table

Create the custom table with Label **Institution Details**, Name **u_institution_details**, and no parent table. Add the following fields:

Field	Type	Purpose
Student Roll Number	Auto Number	Unique student record identifier.
Student Name	Reference – User	Links the student record to a ServiceNow user.
Faculty Name	Reference – User	Links the faculty record to a ServiceNow user.
Branch	Choice	Values: ECE, EEE, CSE.
Email	String	Stores email information.
Phone Number	String	Stores phone information.
Description	Multi String	Stores additional record details.

Milestone 3: Script-Controlled READ ACL

Create an active advanced record READ ACL for **u_institution_details**. Require the bb1 role and use the Branch condition and server-side script to enforce the record-level rule. Administrators retain full access.

Core Script

```
(function () {
// Allow admin users full access
if (gs.hasRole('admin')) {
return true;
}

// User must have the bb1 role
if (!gs.hasRole('bb1')) {
return false;
}

// Allow only EEE branch records
if (current.getValue('branch') == 'EEE') {
return true;
}

return false;
})();
```

Milestones 4–6: CRUD Security and Verification

Date	29 September 2026
Team ID	xxxxxx
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	Documentation

Operation	Required Role	Configuration	Expected Behavior
READ	bb1	Advanced ACL with administrator bypass, role check and Branch = EEE condition.	bb1 users read allowed EEE records; administrators retain full access.
CREATE	bb2	Record CREATE ACL; no additional data condition specified.	Users with bb2 can create records.
WRITE	bb3	Record WRITE ACL; no additional data condition specified.	Users with bb3 can edit records permitted by the access model.
DELETE	bb4	Record DELETE ACL; no additional data condition specified.	Users with bb4 can delete records permitted by the access model.

Verification Procedure

1. Impersonate the EEE User and open the u_institution_details list.
2. Verify that the READ ACL restricts visibility according to the Branch value.
3. Verify CREATE using a user with the required bb2 role.
4. Verify WRITE using a user with the required bb3 role.
5. Verify DELETE using a user with the required bb4 role.
6. Impersonate an administrator and verify full access to all records.
7. Record screenshots/evidence of the list and form behavior.

Conclusion

The project demonstrates record-level security in ServiceNow by combining user roles with a field-value condition. The custom Institution Details table is protected through separate READ, CREATE, WRITE and DELETE ACLs, while administrator access is preserved. The final verification uses impersonation to demonstrate how ACL rules affect list and form access.